

Aussie EcoLens AWS Console HD Implementation Guide

Manual AWS setup for Cognito, S3, Lambda inline code, API Gateway, SNS, and deployment checks.

Use this guide when you want to paste Lambda source manually in the AWS console instead of deploying one zip per Lambda function. A shared Lambda Layer is still required for third-party Python packages; AWS inline code cannot import Google Cloud client libraries without a layer.

1. Target Architecture

- AWS Cognito protects the React UI and every REST API route using a JWT authorizer.
- S3 receives browser uploads using presigned PUT URLs and emits ObjectCreated events.
- Lambda presign_upload reserves SHA-256 checksums before upload to prevent duplicate storage.
- Lambda ingest_dispatcher copies the S3 object to GCP Cloud Storage and publishes a Pub/Sub processing job.
- GCP Cloud Run creates thumbnails, extracts 1 frame per second for videos, runs the ML model, writes Firestore metadata, and triggers tag notifications.
- API Gateway routes query, thumbnail lookup, query-by-file, bulk tag editing, delete, and notification watch operations.

2. Manual Console Values To Prepare

Value	Example	Where used
AWS region	ap-southeast-2	Cognito, S3, Lambda, API Gateway, SNS
S3 upload bucket	fit5225-aussie-ecolens-uploads	UPLOAD_BUCKET
GCP project id	your-gcp-project-id	GCP_PROJECT_ID
GCP processing bucket	fit5225-aussie-ecolens-processing	GCP_BUCKET
Pub/Sub topic	projects/PROJECT/topics/media-processing	PUBSUB_TOPIC
Frontend origin	http://localhost:5174 or deployed HTTPS URL	CORS_ORIGIN and S3 CORS
Cloud Run query endpoint	https://.../query-file	QUERY_FILE_PROCESSOR_URL

3. Cognito Setup

1. Open AWS Cognito, create a User Pool, and choose email sign-in.
2. Required attributes: email, given_name, family_name. Keep email verification enabled.
3. Create an app client for a single-page application. Do not create a client secret.
4. Record the User Pool ID and Client ID for frontend .env and API Gateway JWT authorizer.
5. In the frontend, set VITE_COGNITO_USER_POOL_ID and VITE_COGNITO_CLIENT_ID.

4. S3 Upload Bucket

1. Create the upload bucket and block all public access.
2. Enable server-side encryption with SSE-S3.

3. Add CORS allowing PUT and GET from your frontend origin.
4. Do not make uploaded objects public. The app uses presigned URLs and GCP signed URLs.

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["PUT", "GET"],
    "AllowedOrigins": ["https://YOUR_FRONTEND_DOMAIN"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3000
  }
]
```

5. Shared Lambda Dependency Layer

Manual inline Lambda code can only import the Python standard library and runtime-included AWS SDK packages. These functions also import Google Cloud SDK packages and requests, so create one shared layer and attach it to every Python Lambda.

1. Open AWS CloudShell in the same region.
2. Run the commands below to build a Python 3.12 compatible layer.
3. In Lambda, create a layer named aussie-ecolens-python-deps from layer.zip.
4. Attach the layer to all six Lambda functions.

```
mkdir -p layer/python
python3 -m pip install -t layer/python boto3 google-cloud-firestore google-cloud-storage google-cloud-pubsub google-auth requests
cd layer
zip -r ../layer.zip python
```

6. IAM Role For Lambda

Create one execution role for all functions for coursework simplicity. For a stricter HD explanation, mention that production would split these permissions per function.

Logs:
AWSLambdaBasicExecutionRole

Inline policy:
s3:GetObject, s3:PutObject, s3:DeleteObject, s3:HeadObject on
arn:aws:s3:::YOUR_UPLOAD_BUCKET/*
secretsmanager:GetSecretValue on the GCP service-account secret ARN
sns:CreateTopic, sns:Publish, sns:Subscribe, sns:Unsubscribe on
arn:aws:sns:REGION:ACCOUNT_ID:aussie-ecolens-*

7. Store GCP Service Account In AWS Secrets Manager

1. In GCP, create a service account for AWS dispatcher/API access.
2. Grant it Storage Object Admin on the GCP processing bucket, Datastore User for Firestore, and Pub/Sub Publisher on the media-processing topic.
3. Download the JSON key.
4. In AWS Secrets Manager, create a secret named gcp-firestore-service-account and paste the full JSON as plaintext.

8. Create Lambda Functions Manually

Create each Lambda in the AWS console with runtime Python 3.12, architecture x86_64, handler handler.handler, the shared IAM role, and the shared dependency layer. In the inline editor, create handler.py for the function handler and a common folder containing the four common helper files.

Lambda	Local handler source	Timeout	Memory
presign_upload	aws/lambda/presign_upload/handler.py	60 s	512 MB
ingest_dispatcher	aws/lambda/ingest_dispatcher/handler.py	300 s	1024 MB
query_api	aws/lambda/query_api/handler.py	60 s	512 MB
tag_api	aws/lambda/tag_api/handler.py	60 s	512 MB
delete_api	aws/lambda/delete_api/handler.py	60 s	512 MB
notification_api	aws/lambda/notification_api/handler.py	60 s	512 MB

Common files to create under each Lambda's common/ folder:

- aws/lambda/common/auth.py
- aws/lambda/common/gcp.py
- aws/lambda/common/response.py
- aws/lambda/common/storage.py

9. Lambda Environment Variables

Variable	Value
UPLOAD_BUCKET	Your S3 upload bucket
GCP_SECRET_ID	gcp-firestore-service-account
GCP_BUCKET	Your GCP processing bucket
GCP_PROJECT_ID	Your GCP project id
PUBSUB_TOPIC	projects/PROJECT/topics/media-processing
CORS_ORIGIN	Your frontend origin
AWS_REGION	ap-southeast-2
SNS_TOPIC_PREFIX	arn:aws:sns:REGION:ACCOUNT_ID:aussie-ecolens-
QUERY_FILE_PROCESSOR_URL	Cloud Run URL ending in /query-file; required on query_api

10. S3 Trigger

1. Open the S3 upload bucket, Properties, Event notifications.
2. Create event name ingest-dispatcher-trigger.
3. Prefix: uploads/. Event type: All object create events.

4. Destination: Lambda function ingest_dispatcher.
5. If AWS asks for permission, allow S3 to invoke the Lambda.

11. API Gateway HTTP API

1. Create an HTTP API.
2. Create Lambda integrations for presign_upload, query_api, tag_api, delete_api, and notification_api.
3. Create a JWT authorizer using Cognito issuer https://cognito-idp.REGION.amazonaws.com/USER_POOL_ID and audience CLIENT_ID.
4. Attach the JWT authorizer to every route below. Do not leave routes public.
5. Enable CORS for Authorization and Content-Type headers from your frontend origin.

Route	Lambda integration	Purpose
POST /uploads/presign	presign_upload	Checksum dedupe and presigned S3 upload
GET /media/{mediald}	query_api	Poll upload processing status
POST /query/tags	query_api	AND query with minimum tag counts
POST /query/species	query_api	Species query without counts
POST /query/thumbnail	query_api	Thumbnail URL to full image URL
POST /query/file	query_api	Temporary uploaded query image
POST /media/tags/bulk	tag_api	Bulk add/remove tags
POST /media/delete	delete_api	Delete media, thumbnails, indexes, DB rows
POST /notifications/watch	notification_api	Create tag email watch
POST /notifications/unwatch	notification_api	Disable tag watch

12. SNS Notifications

- notification_api creates SNS topics idempotently for watched tags using the configured SNS_TOPIC_PREFIX.
- Users must confirm the SNS subscription email before notifications can arrive.
- Cloud Run can publish directly to SNS if AWS credentials are configured, or call an AWS notification endpoint if you choose that variant.

13. Frontend Deployment

```
cd frontend
cp ../.env.example .env
# Set VITE_COGNITO_USER_POOL_ID, VITE_COGNITO_CLIENT_ID, VITE_API_BASE_URL
npm install
npm run build
```

- For demo, you can run `npm run dev -- --host 127.0.0.1 --port 5173`.
- For hosted demo, deploy frontend/dist through Amplify Hosting or S3 + CloudFront.

14. HD Demo Test Order

1. Create a new Cognito user with email, first name, last name, and password. Verify email.
2. Sign in and confirm unauthenticated users cannot call API Gateway routes.
3. Upload a test image. Show checksum calculation, successful presign, S3 upload, status polling, and READY result.
4. Upload the same file again. Show duplicate blocked before upload storage is wasted.
5. Upload a video. Explain 1 frame per second extraction in Cloud Run.
6. Run tag count query such as felis_catus:1 and a multi-tag AND query.
7. Run species query, thumbnail URL query, and query-by-file.
8. Select multiple results, bulk add a tag, bulk remove a tag, then delete selected media.
9. Create a watch for a tag and confirm SNS subscription email.

15. Common Demo Failures And Fixes

Symptom	Likely cause	Fix
401/403 from API	JWT authorizer missing or wrong audience	Check Cognito Client ID and Authorization header
Lambda cannot import google.cloud	Layer missing	Attach shared dependency layer to every Lambda
S3 upload CORS error	Frontend origin not allowed	Update bucket CORS and API CORS
Query by file fails	QUERY_FILE_PROCESSOR_URL missing	Set Cloud Run /query-file URL on query_api
No processing after upload	S3 trigger missing or metadata absent	Check ObjectCreated trigger and presign metadata
SNS watch creates no email	Subscription not confirmed	Open email and confirm subscription

16. Final Marking Checklist

- Cognito sign-up, email verification, sign-in, and sign-out work.
- All API Gateway routes are JWT-protected.
- S3 bucket is private, encrypted, and CORS-configured.
- All six Lambda functions use handler.handler and include common helper files.
- Shared dependency layer is attached to all Lambdas.
- Checksum dedupe blocks repeated uploads.
- Cloud Run writes tags, file type, full URL, thumbnail URL, and status to Firestore.
- Queries implement logical AND with minimum counts.
- Bulk tag edit, delete, thumbnail lookup, query-by-file, and notifications are demo-ready.

Appendix A. Local Files To Copy Into AWS Console

Paste these files from your workspace into the Lambda console. Use the exact filenames and folder names shown.

Console path	Workspace source
handler.py	aws/lambda/presign_upload/handler.py
handler.py	aws/lambda/ingest_dispatcher/handler.py
handler.py	aws/lambda/query_api/handler.py
handler.py	aws/lambda/tag_api/handler.py
handler.py	aws/lambda/delete_api/handler.py
handler.py	aws/lambda/notification_api/handler.py
common/auth.py	aws/lambda/common/auth.py
common/gcp.py	aws/lambda/common/gcp.py
common/response.py	aws/lambda/common/response.py
common/storage.py	aws/lambda/common/storage.py